



Tutorials ▾

References ▾

Exercises ▾

Certificates ▾



Sign In

[HTML](#) [CSS](#) [JAVASCRIPT](#) [SQL](#) [PYTHON](#) [JAVA](#) [PHP](#) [HOW TO](#) [W3.CSS](#) [C](#) [C++](#) [C#](#) [BOO](#) [➤](#)

Python Dictionaries ▾

Python If...Else

Python Match

Python While Loops

Python For Loops

Python Functions

Python Decorators

Python Range

Python Lambda

Python Arrays

Python OOP

Python Classes/Objects

Python Inheritance

Python Iterators

Python Polymorphism

Python Scope

Python Modules

Python Dates

Python Math

Python JSON

Python RegEx

Python PIP

Python Try...Except

Python String Formatting

Python None

Python User Input

Python VirtualEnv

File Handling

Python File Handling

Python Read Files

Python Write/Create Files

Python Delete Files

Python Modules

[< Previous](#)[Next >](#)

What is a Module?

Consider a module to be the same as a code library.

A file containing a set of functions you want to include in your application.

Create a Module

To create a module just save the code you want in a file with the file extension `.py`:

Example

[Get your own Python Server](#)

Save this code in a file named `mymodule.py`

```
def greeting(name):  
    print("Hello, " + name)
```

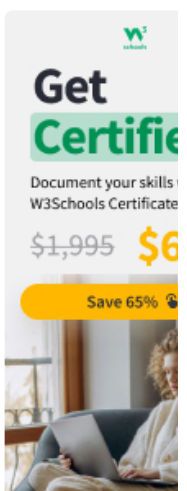
Use a Module

Now we can use the module we just created, by using the `import` statement:

Example

Import the module named mymodule, and call the greeting function:

```
import mymodule
```

COLOR
PICKER

```
mymodule.greeting("Jonathan")
```

[Run Example »](#)[AI](#)

Note: When using a function from a module, use the syntax:
module_name.function_name.

Variables in Module

The module can contain functions, as already described, but also variables of all types (arrays, dictionaries, objects etc):

Example

Save this code in the file `mymodule.py`

```
person1 = {  
    "name": "John",  
    "age": 36,  
    "country": "Norway"  
}
```

Example

Import the module named mymodule, and access the person1 dictionary:

```
import mymodule  
  
a = mymodule.person1["age"]  
print(a)
```

[Run Example »](#)[R](#)

ADVERTISEMENT

[REMOVE ADS](#)

Naming a Module

You can name the module file whatever you like, but it must have the file extension `.py`

Re-naming a Module

You can create an alias when you import a module, by using the `as` keyword:

Example

Create an alias for `mymodule` called `mx` :

```
import mymodule as mx

a = mx.person1["age"]
print(a)
```

[Run Example »](#)

Built-in Modules

There are several built-in modules in Python, which you can import whenever you like.

Example

Import and use the `platform` module:

```
import platform

x = platform.system()
print(x)
```

[Try it Yourself »](#)

Using the dir() Function

There is a built-in function to list all the function names (or variable names) in a module. The `dir()` function:

Example

List all the defined names belonging to the platform module:

```
import platform

x = dir(platform)
print(x)
```

Try it Yourself »

Note: The `dir()` function can be used on *all* modules, also the ones you create yourself.

Import From Module

You can choose to import only parts from a module, by using the `from` keyword.

Example

The module named `mymodule` has one function and one dictionary:

```
def greeting(name):
    print("Hello, " + name)

person1 = {
    "name": "John",
    "age": 36,
    "country": "Norway"
}
```

Example

Import only the person1 dictionary from the module: